

07-自定义指令+插槽+比特课程案例

一、自定义指令

1. 基本使用

1.1 指令介绍

- 内置指令：v-model、v-for、v-bind、v-on... 这都是Vue给咱们内置的一些指令，可以直接使用
- 自定义指令：同时Vue也支持让开发者，自己注册一些指令。这些指令被称为自定义指令

1.2 作用

封装一段 **公共的DOM操作** 代码，便于复用

1.3 语法

1. 注册

```
1 // main.js 中
2 app.directive('指令名', {
3   // 元素挂载后(成为真实DOM) 自动触发一次
4   // 自动执行
5   mounted(el) {
6     // el: 指令所在的DOM元素
7   }
8 })
```

2. 使用

```
1 <p v-指令名></p>
```

1.4 代码示例

需求：当页面加载时, 让元素获取焦点

```
1 // main.js
2
3 // 注册全局指令
```

```
4 app.directive('focus', {
5   mounted(el) {
6     console.log(el) // input 元素
7     // 聚焦
8     el.focus()
9   }
10 })
```

App.vue

```
1 <script setup></script>
2
3 <template>
4   <div class="app">
5     <input type="text" v-focus />
6   </div>
7 </template>
```

1.5 总结

1. 自定义指令的作用是什么？

答：封装一段公共的 **DOM操作** 的代码

2. 使用自定义指令的步骤是哪两步？

答：先注册、后使用

3. 指令配置选项中的 mounted 钩子何时执行？

答：元素 **挂载后** (成为DOM树的一部分时) 自动执行

2. 绑定数据

2.1 需求

实现一个 color 指令：传入不同的颜色, 给标签设置文字颜色

2.2 语法

1.在绑定指令时，可以通过“等号”的形式为指令 绑定 具体的参数值

```
1 <div v-color="colorStr">Some Text</div>
```

2.通过 binding.value 可以拿到指令值，指令值修改会 触发 **updated 钩子**

```
1 app.directive('指令名', {
2   // 挂载后自动触发一次
3   mounted(el, binding) { },
4   // 数据更新, 每次都会执行
5   updated(el, binding) { }
6 })
```

2.3 代码示例

```
1 // main.js
2 app.directive('color', {
3   mounted(el, binding) {
4     el.style.color = binding.value
5   },
6   updated(el, binding) {
7     el.style.color = binding.value
8   }
9 })
```

App.vue

```
1 <script setup>
2   import { ref } from 'vue'
3   // 颜色
4   const colorStr = ref('red')
5 </script>
6
7 <template>
8   <p v-color="colorStr"></p>
9 </template>
```

2.4 简化形式

对于自定义指令来说, 一个很常见的情况是仅仅需要在 `mounted` 和 `updated` 上实现相同的行
为。这种情况下我们可以直接用一个函数来定义指令, 如下所示:

```
1 app.directive('color', (el, binding) => {
2   // 这会在 `mounted` 和 `updated` 时都调用
3   el.style.color = binding.value
4 })
```

3. 案例图片懒加载

3.1 场景

实际开发过程中，如果项目中图片过多，我们不会一次加载所有的图片，而是当图片出现在可视区的时候才去加载，比如京东、淘宝等都采用了这种方案。

3.2 解决方案

封装一个 **v-lazyload** 自定义指令，实现图片懒加载，从而节省资源、提高性能

3.3 准备代码

在 **App.vue** 中

```
1  <script setup>
2    const imgList = [
3
4      'https://img1.baidu.com/it/u=14492133,1259363498&fm=253&fmt=auto&app=138&f=JPEG?w=667&h=500',
5
6      'https://img0.baidu.com/it/u=1764531212,1643995922&fm=253&fmt=auto&app=120&f=JPEG?w=750&h=500',
7
8      'https://img1.baidu.com/it/u=3461494820,2726880132&fm=253&fmt=auto&app=138&f=JPEG?w=773&h=500',
9
10     'https://img1.baidu.com/it/u=2991964469,2851730176&fm=253&fmt=auto&app=138&f=JPEG?w=889&h=500',
11
12     'https://img2.baidu.com/it/u=1519104236,3241953583&fm=253&fmt=auto&app=138&f=JPEG?w=781&h=500',
13
14     'https://img0.baidu.com/it/u=3431675376,3243768390&fm=253&fmt=auto&app=138&f=JPEG?w=712&h=447',
15
16     'https://img1.baidu.com/it/u=2111075854,406597938&fm=253&fmt=auto&app=138&f=JPEG?w=888&h=500',
17
18     'https://img0.baidu.com/it/u=1615464091,2840643412&fm=253&fmt=auto?w=945&h=605',
19
20     'https://img0.baidu.com/it/u=3926979850,631936366&fm=253&fmt=auto&app=120&f=JPEG?w=785&h=500',
21
22     'https://img1.baidu.com/it/u=469866567,781924764&fm=253&fmt=auto&app=120&f=JPEG?w=750&h=500'
```

```

13   ]
14   </script>
15
16   <template>
17     <div class="container">
18       
24     </div>
25   </template>
26
27   <style lang="scss">
28     * {
29       margin: 0;
30     }
31     .container {
32       width: 600px;
33       display: flex;
34       flex-direction: column;
35       margin: 0 auto;
36     }
37   </style>

```

3.4 认识 IntersectionObserver

- MDN: <https://developer.mozilla.org/zh-CN/docs/Web/API/IntersectionObserver>
- 掘金: <https://article.juejin.cn/post/6844903560434417677>

3.5 完整代码

在 `main.js` 中

代码块

```

1  // 判断元素有没有出现在可视区, 用到一个新的API, IntersectionObserver(交叉监视器)
2  app.directive('lazyload', (el, binding) => {
3    // el: img标签
4    // binding.value: 实际要展示的图片路径
5    // 原理: 判断 el 有没有出现在可视区, 如果是的话, 才去给 el 设置 src 属性
6
7    // 实例化 IntersectionObserver 对象
8    const io = new IntersectionObserver(([entry]) => {
9      // entry: 交叉状态对象
10      if (entry.isIntersecting) {

```

```

11      // 说明图片与可视区发生了交叉，也就是图片出现在了可视区
12      el.src = binding.value
13
14      // 监听图片加载错误事件
15      el.addEventListener('error', (error) => {
16          console.log('图片加载失败, ', error)
17      })
18
19      // 停止监听
20      io.unobserve(el)
21      // 关闭监听器
22      io.disconnect()
23  }
24  })
25
26      // 开启监视
27      io.observe(el)
28  })

```

在 App.vue 中

代码块

```

1  <template>
2    <div class="container">
3      <img
4        v-for="(url, index) in imgList"
5        :key="index"
6        v-lazyload="url" />
7    </div>
8  </template>

```

二、插槽

1. 什么是插槽

允许开发者对组件的结构自定义，本质就是一个占位，使用组件的时候传入指定的标签结构，从而做一个替换

2. 分类

- 默认/匿名插槽
- 具名插槽
- 作用域插槽

3. 默认插槽

3.1 作用

让组件的部分 **结构** 支持 **自定义**

3.2 需求

让折叠面板组件的内容, 支持自定义

3.3 插槽的基本语法

1. 组件内需要定制的结构部分, 使用 **slot** 标签占位
2. 使用组件时, 把组件写成**双标签**, 包裹需要展示的标签结构, 从而替换掉slot

3.4 代码示例

bit-panel.vue

```
1  <script setup>
2    import { ref } from 'vue'
3    const visible = ref(false)
4  </script>
5
6  <template>
7    <div class="panel">
8      <div
9        class="title"
10       :style="{
11         borderBottomColor: visible ? 'transparent' : '#ccc'
12       }">
13        <h4>自由与爱情</h4>
14        <span
15          class="btn"
16          @click="visible = !visible">
17          收起
18        </span>
19      </div>
20      <div
21        class="container"
22        v-show="visible">
23        <p>生命诚可贵,</p>
24        <p>爱情价更高.</p>
25        <p>若为自由故,</p>
26        <p>两者皆可抛.</p>
27      </div>
28    </div>
```

```

29 </template>
30
31 <style lang="scss" scoped>
32   .panel {
33     margin-bottom: 10px;
34     .title {
35       display: flex;
36       justify-content: space-between;
37       align-items: center;
38       padding: 0 1em;
39       border: 1px solid #ccc;
40     }
41     .title h4 {
42       line-height: 2;
43       margin: 0;
44     }
45     .container {
46       border: 1px solid #ccc;
47       padding: 10px 15px;
48     }
49     .btn {
50       cursor: pointer;
51     }
52   }
53 </style>

```

App.vue

```

1 <script setup>
2   import BitPanel from './components/bit-panel.vue'
3 </script>
4
5 <template>
6   <h3>折叠面板</h3>
7   <bit-panel />
8 </template>
9
10 <style lang="scss">
11   * {
12     margin: 0;
13     padding: 0;
14   }
15   body {
16     background: #ccc;
17   }

```



```

18   #app {
19     width: 400px;
20     margin: 30px auto;
21     padding: 1em 2em 2em;
22     background: #fff;
23   }
24   #app h3 {
25     margin-bottom: 20px;
26     text-align: center;
27   }
28 </style>

```

3.5 完整代码

在 `bit-panel.vue` 中

代码块

```

1
2 <template>
3   <div class="panel">
4     <div
5       class="title"
6       :style="{
7         borderBottomColor: visible ? 'transparent' : '#ccc'
8       }">
9       <h4>自由与爱情</h4>
10      <span
11        class="btn"
12        @click="visible = !visible">
13        收起
14      </span>
15    </div>
16    <div
17      class="container"
18      v-show="visible">
19      <!-- 在组件标签不确定的位置，用 slot 占位 -->
20      <!--
21        slot: Vue的内置组件，也是一个抽象组件(不会出现在真实DOM树)，拿来就用，代表是一个
占位符
22        -->
23      <slot></slot>
24    </div>
25  </div>
26 </template>

```

在 App.vue 中

代码块

```
1  <template>
2    <h3>折叠面板</h3>
3    <!-- 展示一首诗 -->
4    <bit-panel>
5      <p>生命诚可贵，</p>
6      <p>爱情价更高。</p>
7      <p>若有自由故，</p>
8      <p>两者皆可抛。</p>
9    </bit-panel>
10
11   <!-- 展示2张图片 -->
12   <bit-panel>
13     <div class="img-box">
14       
18       
22     </div>
23   </bit-panel>
24 </template>
25
26 <style lang="scss">
27   .img-box {
28     display: flex;
29     flex-direction: column;
30   }
31 </style>
```

3.6 总结

1. 组件内某一部分结构不确定，想要自定义怎么办？

答：插槽

2. 插槽的步骤分为哪几步？

答：两步；先占位、后传入

4. 插槽默认值

4.1 问题

通过插槽完实现内容的自定义, 传什么显示什么, 但是如果不传, 则是空白

能否给插槽设置 默认显示内容 呢?

4.2 解决方案

封装组件时, 可以为 `<slot>` 提供默认内容

4.3 语法

在 `<slot>` 标签内, 放置内容, 作为默认内容

4.4 效果

- 使用组件时, 不传, 则会显示slot的默认内容;

如果不给组件传入插槽, 推荐把组件写成自闭合的单标签, 这样更加简洁

```
<bit-panel />
<bit-panel></bit-panel>
```

- 使用组件时, 传了(想展示什么就传入什么), 则slot整体会被换替换掉, 从而显示传入的

```
<bit-panel>
  <div class="img-box">
    
    
  </div>
</bit-panel>
```

4.5 代码示例

在 `bit-panel.vue` 中

代码块

```
1  <template>
2    <div class="panel">
3      <div
4        class="title"
5        :style="{
6          borderBottomColor: visible ? 'transparent' : '#ccc'
7        }">
8        <h4>自由与爱情</h4>
9        <span
10         class="btn"
11         @click="visible = !visible">
```

```

12     收起
13     </span>
14 </div>
15 <div
16     class="container"
17     v-show="visible">
18     <!--
19     插槽的默认值： slot 标签包裹的内容
20     在使用组件的时候：
21     1) 没有传内容，展示插槽的默认值
22     2) 有传递内容，传递的内容会替换掉整个 slot，展示实际传入的内容
23     -->
24     <slot>
25         <p>白日依山尽，</p>
26         <p>黄河入海流。</p>
27         <p>欲穷千里目，</p>
28         <p>更上一层楼。</p>
29     </slot>
30 </div>
31 </div>
32 </template>

```

在 App.vue 中

```

1  <script setup>
2    import BitPanel from './components/bit-panel.vue'
3  </script>
4
5  <template>
6    <h3>折叠面板</h3>
7    <!-- 第一次使用：没有传递内容，就会展示插槽的默认值 -->
8    <bit-panel />
9    <bit-panel></bit-panel>
10
11    <!-- 第二次使用：传递内容，展示两张图片 -->
12    <bit-panel>
13      <div class="img-box">
14        
15        
16      </div>
17    </bit-panel>
18  </template>
19
20  <style lang="scss">
21    .img-box {

```

```
22     display: flex;
23     flex-direction: column;
24     img {
25         width: 100%;
26     }
27 }
28 </style>
```

4.6 总结

1. 插槽默认内容有什么用？

答：使用组件不传内容时, 防止出现空白

2. 默认内容何时显示？何时不显示？

答：使用组件时, 不传内容, 则显示默认内容; 否则显示传递的内容

5. 具名插槽

5.1 需求

一个组件内有多处结构，需要外部传入标签，进行自定义

自由与爱情	收起
生命诚可贵， 爱情价更高。 若为自由故， 两者皆可抛。	

登鹳雀楼	收起
白日依山尽， 黄河入海流。 欲穷千里目， 更上一层楼。	



上面的折叠面板组件中有 **两处不同**，但是 **默认插槽** 只能 **约束一处内容**，这时怎么办？

5.2 具名插槽语法

- 多个slot，使用name属性区分
- template + #插槽名 来分发相应的标签结构

5.3 代码示例

在 `bit-panel.vue` 中

```
1  <template>
2    <div class="panel">
3      <div
4        class="title"
5        :style="{
6          borderBottomColor: visible ? 'transparent' : '#ccc'
7        }">
8        <h4>
9          <slot name="title">
10             <span>自由与爱情</span>
11          </slot>
12        </h4>
13      </div>
```

```

14         class="btn"
15         @click="visible = !visible">
16         收起
17     </span>
18 </div>
19 <div
20     class="container"
21     v-show="visible">
22     <slot name="body">
23         <p>生命诚可贵，</p>
24         <p>爱情价更高。</p>
25         <p>若为自由故，</p>
26         <p>两者皆可抛。</p>
27     </slot>
28 </div>
29 </div>
30 </template>

```

在 App.vue 中

```

1  <script setup>
2      import BitPanel from './components/bit-panel.vue'
3  </script>
4
5  <template>
6      <h3>折叠面板</h3>
7      <!-- 第一次使用：两处插槽都没有传值，就会展示插槽的默认值 -->
8      <bit-panel />
9      <!-- 第二次使用：展示登鹳雀楼这首诗及内容 -->
10     <bit-panel>
11         <template #title>
12             <span>登鹳雀楼</span>
13         </template>
14         <template #body>
15             <p>白日依山尽，</p>
16             <p>黄河入海流。</p>
17             <p>欲穷千里目，</p>
18             <p>更上一层楼。</p>
19         </template>
20     </bit-panel>
21     <!-- 第三次使用：展示两个女明星+两张图片 -->
22     <bit-panel>
23         <template #body>
24             <div class="img-box">
25                 

```

```

26         
27     </div>
28 </template>
29 <template #title>
30     <b>两个女明星</b>
31 </template>
32 </bit-panel>
33 </template>
34
35 <style lang="scss">
36     .img-box {
37         display: flex;
38         flex-direction: column;
39         img {
40             width: 100%;
41         }
42     }
43     p {
44         line-height: 30px;
45     }
46 </style>

```

5.4 总结

1. 组件内有多处不确定的结构 怎么办?

答：具名插槽

2. 具名插槽的使用语法是什么?

答：1、`<slot name="名字"> 默认内容 </slot>`

2、`<template #名字> 要展示的内容 </template>`

6. 作用域插槽

6.1 作用

带数据的插槽，可以让组件功能更强大、更灵活、复用性更高；

用 slot 占位的同时，还可以给 slot **绑定数据**，将来使用组件时，不仅可以传内容，还能使用 slot 带来的数据

6.2 使用步骤

1. 在template中，通过 #插槽名="变量" 接收，如果没有给 slot 取名，则为默认的名字 default

在 App.vue 中


```

1 <bit-panel>
2   <template #title>
3     <b>两个女明星</b>
4   </template>
5   <template #body="obj">
6     <div class="img-box">
7       {{ obj }}
8       
9       
10    </div>
11  </template>
12 </bit-panel>

```

自由与爱情

收起

{} 默认值为空对象

2. 给相应 slot 标签添加任意自定义属性

比如：

在 **bit-panel.vue** 中

代码块

```
1 <slot a="hello" :b="321"></slot>
```

自由与爱情

收起

{ "a": "hello", "b": 321 }

6.3 总结

1. 作用域插槽的作用是什么？

答：让插槽带数据，使得组件功能更强大、更灵活

2. 作用域插槽的使用步骤是什么？

答：1、slot上绑定 任意自定义属性

2、<template #名字="{ 数据 }"> </template>

6.4 美食案例

6.4.1 效果预览

欢迎来到陕西美食

序号	名称	价格	数量	操作
1	肉夹馍	11	2	查看
2	魏家凉皮	10	1	查看
3	茶叶蛋	2	2	查看
4	冰峰	10	2	查看

序号	名称	价格	数量	操作
1	葫芦头	14	4	删除
2	羊肉泡馍	18	3	删除
3	黑色大窑	5	2	删除

同一个表格组件，整体结构一样，但支持不同的操作方式

6.4.2 静态代码

在 `components/bit-table.vue` 中

代码块

```
1  <script setup></script>
2
3  <template>
4    <table
5      class="bit-table"
6      border="1"
7      cellspacing="0"
8      cellpadding="0">
9      <thead>
10     <tr>
11       <th>序号</th>
12       <th>名称</th>
13       <th>价格</th>
```

```

14         <th>数量</th>
15         <th>操作</th>
16     </tr>
17 </thead>
18 <tbody>
19     <tr>
20         <td>1</td>
21         <td>biangbiang面</td>
22         <td>13</td>
23         <td>2</td>
24         <td>操作</td>
25     </tr>
26 </tbody>
27 </table>
28 </template>
29
30 <style lang="scss" scoped>
31   .bit-table {
32     width: 500px;
33     border-color: #fafdfe;
34     text-align: center;
35   }
36
37   .bit-table thead {
38     background: #9282d3;
39   }
40
41   .bit-table thead tr th {
42     border-width: 0;
43   }
44
45   .bit-table tr {
46     height: 40px;
47   }
48 </style>

```

在 App.vue 中

代码块

```

1 <script setup>
2   import { ref } from 'vue'
3
4   const tableData1 = ref([
5     { id: 139, name: '肉夹馍', price: 11, num: 2 },
6     { id: 340, name: '魏家凉皮', price: 10, num: 1 },
7     { id: 397, name: '茶叶蛋', price: 2, num: 2 },

```

```

8      { id: 262, name: '冰峰', price: 10, num: 2 }
9    ])
10
11    const tableData2 = ref([
12      { id: 340, name: '葫芦头', price: 14, num: 4 },
13      { id: 118, name: '羊肉泡馍', price: 18, num: 3 },
14      { id: 273, name: '黑色大窑', price: 5, num: 2 }
15    ])
16
17  </script>
18
19  <template>
20    <div class="main">
21      <h2>欢迎来到陕西美食</h2>
22    </div>
23  </template>
24
25  <style lang="scss" scoped>
26    .main {
27      display: flex;
28      flex-direction: column;
29      align-items: center;
30    }
31    a {
32      display: block;
33      text-decoration: none;
34      font-weight: bold;
35      color: rgb(206, 21, 21);
36    }
37  </style>

```

代码块

```

1  <script setup>
2    import BitTable from './components/bit-table.vue'
3
4    import { ref } from 'vue'
5
6    const tableData1 = ref([
7      { id: 139, name: '肉夹馍', price: 11, num: 2 },
8      { id: 340, name: '魏家凉皮', price: 10, num: 1 },
9      { id: 397, name: '茶叶蛋', price: 2, num: 2 },
10     { id: 262, name: '冰峰', price: 10, num: 2 }
11   ])
12

```

```
13   const tableData2 = ref([
14     { id: 340, name: '葫芦头', price: 14, num: 4 },
15     { id: 118, name: '羊肉泡馍', price: 18, num: 3 },
16     { id: 273, name: '黑色大窑', price: 5, num: 2 }
17   ])
18
19   const inspect = (row) => {
20     alert(JSON.stringify(row))
21   }
22
23   const del = (row) => {
24     if (window.confirm('确认删除么')) {
25       tableData2.value = tableData2.value.filter((item) => item.id !== row.id)
26     }
27   }
28 </script>
29
30 <template>
31   <div class="main">
32     <h2>欢迎来到陕西美食</h2>
33     <bit-table :data="tableData1">
34       <template #default="{ row }">
35         <span @click="inspect(row)">查看</span>
36       </template>
37     </bit-table>
38     <br />
39     <br />
40     <bit-table :data="tableData2">
41       <template #default="{ row }">
42         <button @click="del(row)">删除</button>
43       </template>
44     </bit-table>
45   </div>
46 </template>
47
48 <style lang="scss" scoped>
49   .main {
50     display: flex;
51     flex-direction: column;
52     align-items: center;
53   }
54   span {
55     display: block;
56     cursor: pointer;
57     font-weight: bold;
58     color: rgb(206, 21, 21);
59   }
```

6.4.3 封装表格组件

在 **bit-table.vue** 中

代码块

```
1  <script setup>
2    const props = defineProps({
3      data: {
4        type: Array,
5        default: () => []
6      }
7    })
8  </script>
9
10 <template>
11   <table
12     class="bit-table"
13     border="1"
14     cellpadding="0"
15     cellspacing="0">
16     <thead>
17       <tr>
18         <th>序号</th>
19         <th>名称</th>
20         <th>价格</th>
21         <th>数量</th>
22         <th>操作</th>
23       </tr>
24     </thead>
25     <tbody>
26       <tr
27         v-for="(item, index) in props.data"
28         :key="item.id">
29         <td>{{ index + 1 }}</td>
30         <td>{{ item.name }}</td>
31         <td>{{ item.price }}</td>
32         <td>{{ item.num }}</td>
33         <td>
34           <slot :i="index"></slot>
35         </td>
36       </tr>
37     </tbody>
38   </table>
39 </template>
```

```

40
41 <style lang="scss" scoped>
42   .bit-table {
43     width: 500px;
44     border-color: #fafdfe;
45     text-align: center;
46   }
47
48   .bit-table thead {
49     background: #9282d3;
50   }
51
52   .bit-table thead tr th {
53     border-width: 0;
54   }
55
56   .bit-table tr {
57     height: 40px;
58   }
59 </style>

```

6.4.4 使用表格组件

在 **App.vue** 中

代码块

```

1  <!-- @format -->
2
3  <script setup>
4    import BitTable from './components/bit-table.vue'
5    import { ref } from 'vue'
6
7    const tableData1 = ref([
8      { id: 139, name: '肉夹馍', price: 11, num: 2 },
9      { id: 340, name: '魏家凉皮', price: 10, num: 1 },
10     { id: 397, name: 'biangbiang面', price: 14, num: 2 },
11     { id: 262, name: '冰峰', price: 10, num: 2 }
12   ])
13
14   const tableData2 = ref([
15     { id: 340, name: '葫芦头', price: 14, num: 4 },
16     { id: 118, name: '羊肉泡馍', price: 18, num: 3 },
17     { id: 273, name: '黑色大窑', price: 5, num: 2 }
18   ])
19
20   // 查看

```

```
21     const preview = (index) => {
22         alert(JSON.stringify(tableData1.value[index]))
23     }
24
25     // 删除
26     const del = (index) => {
27         if (window.confirm('确认删除么?')) {
28             tableData2.value.splice(index, 1)
29         }
30     }
31 </script>
32
33 <template>
34     <div class="main">
35         <h2>欢迎来到陕西美食</h2>
36         <bit-table :data="tableData1">
37             <template #default="{ i }">
38                 <a
39                     href="javascript:;"
40                     @click="preview(i)"
41                     >查看</a>
42             >
43             </template>
44         </bit-table>
45         <br />
46         <br />
47         <bit-table :data="tableData2">
48             <template #default="{ i }">
49                 <button @click="del(i)">删除</button>
50             </template>
51         </bit-table>
52     </div>
53 </template>
54
55 <style lang="scss" scoped>
56     .main {
57         display: flex;
58         flex-direction: column;
59         align-items: center;
60     }
61     a {
62         display: block;
63         text-decoration: none;
64         font-weight: bold;
65         color: rgb(206, 21, 21);
66     }
67 </style>
```


三、综合案例

1. 整体效果

比特课程列表

序号	名称	类别	封面	特点
1	C语言刷题	C语言		通俗易懂 上手快
2	C++系统就业课	C++		全面 高效
3	Java系统就业课	Java		牛 全覆盖
4	测试开发系统就业课	测试		爽爆了 干货满满

2. 封装 Table 组件

2.1 静态代码

在 components/bit-table.vue 中

```
1  <script setup></script>
2
3  <template>
4    <table class="bit-table">
5      <thead>
6        <tr>
7          <th>序号</th>
8          <th>名称</th>
9          <th>类别</th>
10         <th>封面</th>
```

```

11         <th class="feature">特点</th>
12     </tr>
13 </thead>
14 <tbody>
15     <tr>
16         <td>1</td>
17         <td>C语言刷题</td>
18         <td>C</td>
19         <td>
20             
23         </td>
24         <td class="feature">特点</td>
25     </tr>
26
27     <tr>
28         <td>1</td>
29         <td>C语言刷题</td>
30         <td>C</td>
31         <td>
32             
35         </td>
36         <td class="feature">特点</td>
37     </tr>
38 </tbody>
39 </table>
40 </template>
41
42 <style lang="scss">
43     .bit-table {
44         width: 1000px;
45         border-spacing: 0;
46         margin: 20px auto;
47         text-align: center;
48         font-family: Microsoft YaHei UI;
49         .cover {
50             height: 130px;
51             vertical-align: middle;
52         }
53         th {
54             background: #f3f5f7;
55         }

```

```
56
57     td,
58     th {
59         padding: 12px 6px;
60         border-bottom: 3px solid rgb(245, 108, 108);
61     }
62     td.feature {
63         width: 210px;
64     }
65 }
66 </style>
```

2.2 插槽改进

在 `components/bit-table.vue` 中

```
1  <script setup>
2      const props = defineProps({
3          data: {
4              type: Array,
5              default: () => []
6          }
7      })
8  </script>
9
10 <template>
11     <table class="bit-table">
12         <thead>
13             <tr>
14                 <slot name="head"></slot>
15             </tr>
16         </thead>
17         <tbody>
18             <tr
19                 v-for="(item, index) in props.data"
20                 :key="item.id">
21                 <slot
22                     :row="item"
23                     :i="index"></slot>
24             </tr>
25         </tbody>
26     </table>
27 </template>
```

3. 封装 Feature 组件

新建 `components/bit-feature.vue` 组件

3.1 静态结构

代码块

```
1  <script setup></script>
2
3  <template>
4    <div class="bit-feature">
5      <input
6        class="ipt"
7        type="text"
8        placeholder="请输入特点" />
9      <div class="feature">
10       <span>易上手</span>
11       <span>全覆盖</span>
12     </div>
13   </div>
14 </template>
15
16 <style lang="scss" scoped>
17   .bit-feature {
18     cursor: pointer;
19     .ipt {
20       box-sizing: border-box;
21       width: 180px;
22       height: 30px;
23       padding: 6px;
24       border: 1px solid #d0d0d0;
25       color: #888;
26       appearance: none;
27       outline: none;
28       &::placeholder {
29         font-size: 12px;
30         color: #888;
31       }
32     }
33     .feature {
34       span {
35         display: inline-block;
36         height: 24px;
37         line-height: 24px;
38         margin: 3px;
39         padding: 0 10px;
```

```
40     font-size: 12px;
41     border-radius: 6px;
42     background: #000;
43     color: #409eff;
44     background: rgb(216.8, 235.6, 255);
45   }
46 }
47 }
48 </style>
```

3.2 定义聚焦指令

在 `main.js` 中

代码块

```
1 app.directive('focus', {
2   mounted(el) {
3     el.focus()
4   }
5 })
```

3.3 处理交互

代码块

```
1 <script setup>
2   import { ref } from 'vue'
3   const model = defineModel()
4
5   // 控制输入框是否显示
6   const visible = ref(false)
7   const f = ref('')
8
9   // 显示
10  const show = () => {
11    visible.value = true
12  }
13
14  // 隐藏
15  const hide = () => {
16    visible.value = false
17    f.value = ''
18  }
19
20  // 新增
```

```

21     const add = () => {
22       if (f.value) {
23         model.value.push(f.value)
24       }
25       hide()
26     }
27   </script>
28
29   <template>
30     <div class="bit-feature">
31       <input
32         v-focus
33         v-if="visible"
34         v-model.trim="f"
35         class="ipt"
36         type="text"
37         placeholder="请输入特点"
38         @blur="hide"
39         @keydown.enter="add" />
40       <div
41         class="feature"
42         v-else
43         @dblclick="show">
44         <span
45           v-for="(f, index) in model"
46           :key="index"
47           >{{ f }}</span>
48       </div>
49     </div>
50   </div>
51 </template>

```

4. App完整代码

代码块

```

1   <script setup>
2     import { ref } from 'vue'
3     import BitTable from './components/bit-table.vue'
4     import BitFeature from './components/bit-feature.vue'
5
6     // 课程列表
7     const courseList = ref([
8       {
9         id: 101001,
10        title: 'C语言刷题',

```

```

11     cover:
12         'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/ffec5c1ce4b0403b163e3865.jpg',
13     feature: ['通俗易懂', '上手快'],
14     type: 'C语言'
15 },
16 {
17     id: 101002,
18     title: 'C++系统就业课',
19     cover:
20         'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/0006cf36e4b05af5acbaeb35.jpg',
21     feature: ['全面', '高效'],
22     type: 'C++'
23 },
24 {
25     id: 101003,
26     title: 'Java系统就业课',
27     cover:
28         'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/0006cf57e4b05af5acbaeb38.jpg',
29     feature: ['牛', '全覆盖'],
30     type: 'Java'
31 },
32 {
33     id: 101004,
34     title: '测试开发系统就业课',
35     cover:
36         'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/0006cf08e4b05af5acbaeb32.jpg',
37     feature: ['爽爆了', '干货满满'],
38     type: '测试'
39 }
40 ])
41
42 // 删除
43 const del = (i) => {
44     if (window.confirm('确认删除么?')) {
45         courseList.value.splice(i, 1)
46     }
47 }
48 </script>
49
50 <template>
51     <h1>比特就业课课程列表</h1>
52     <bit-table :data="courseList">
53         <template #thead>

```

```
54     <th>序号</th>
55     <th>类别</th>
56     <th>名称</th>
57     <th>封面</th>
58     <th>特点</th>
59     <th>操作</th>
60 </template>
61 <template #default="{ row, i }">
62     <td>{{ i + 1 }}</td>
63
64     <td>{{ row.type }}</td>
65     <td>{{ row.title }}</td>
66     <td>
67         
71     </td>
72     <td class="feature">
73         <bit-feature v-model="row.feature" />
74     </td>
75     <td>
76         <button @click="del(i)">删除</button>
77     </td>
78 </template>
79 </bit-table>
80 </template>
81
82 <style lang="scss">
83     h1 {
84         text-align: center;
```